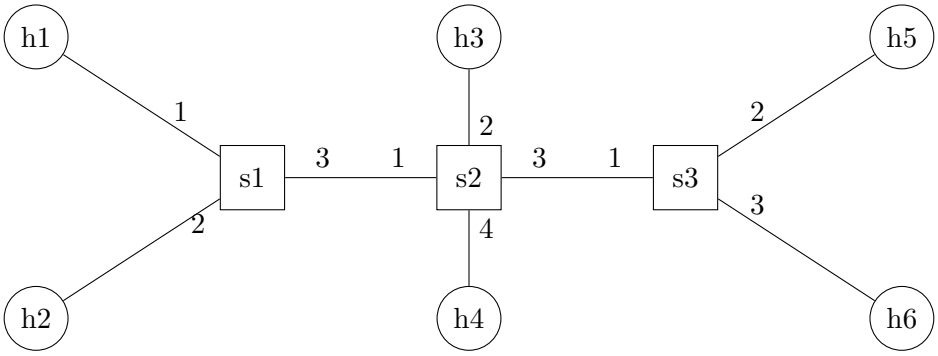
Software Defined Networking
19 luglio 2022

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Negli switch sono preinstallate le regole in Figura 1.

L'host h1 porta 1234 invia un datagramma UDP verso h4 porta 5678.

1. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

> **Soluzione:** S1 invia un messaggio packet-in e riceve in risposta una flow-mod

2. Scrivere con quale TTL il pacchetto arriva alla destinazione.

> **Soluzione:** Il controllore non inoltra il pacchetto che, quindi, viene perso.

L'host h1 porta 1234 invia un altro datagramma UDP verso h4 porta 5678.

3. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

> **Soluzione:** S1 usa la nuova regola e non invia messaggi al controllore

L'host h1 porta 1234 apre una connessione TCP verso h4 porta 5678.

4. Scrivere quali host collegati ad s1 ricevono il pacchetto e con quale TTL.

Soluzione: S1 usa la nuova regola e invia il pacchetto a s2

5. L'host che riceve la richiesta di apertura, risponde. Scrivere quali host collegati ad s1 ricevono il pacchetto di risposta e con quale TTL.

Soluzione: h1 con TTL = 199

6. Scrivere la regola da aggiungere in s2 per fare in modo che le richieste http verso h3 siano inviate ad un web server in esecuzione su h5.

Soluzione:

Priority	Match	Action
20	eth_type = IPv4 ipv4_dst = IP3 ip_proto = TCP tcp_dst = 80	SetField(eth_dst=MAC5) SetField(ipv4_dst=IP5) Output(3)

Switch s1		
Priority	Match	Action
5	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto=UDP	Output(CONTROLLER)
15	eth_dst = MAC2	Output(2)
15	eth_dst = MAC1	Output(1)

Switch s2		
Priority	Match	Action
5	*	DecTTL Output(FLOOD)
10	eth_type = IPv4 ip_proto = TCP	Output(CONTROLLER)
10	eth_type = IPv4 ip_proto = UDP	Output(FLOODING)

Switch s3		
Priority	Match	Action
5	*	DecTTL Output(FLOOD)
15	eth_dst = MAC5	Output(2)
15	eth_dst = MAC6	Output(3)

Figura 1: Regole iniziali

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp =
   pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (pkt_udp is not None) and (datapath.dpid == 1):
        match = parser.OFPMatch(
            eth_src=pkt_eth.src, eth_dst=pkt_eth.dst )
        actions = [
            parser.OFPDecTTL(), parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(
            ofproto.OFPIT_APPLY_ACTIONS, actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=20,
            match=match, instructions=inst)
        datapath.send_msg(mod)
        return

    if (pkt_tcp is not None) and (datapath.dpid == 2):
        match = parser.OFPMatch(
            eth_dst=pkt_eth.src, eth_src=pkt_eth.dst,
            eth_type=ETH_TYPE_IP, ip_proto=IPPROTO_TCP)
        actions = [
            parser.OFPDecTTL(),
            parser.OFPActionOutput(in_port) ]
        inst = [parser.OFPInstructionActions(
            ofproto.OFPIT_APPLY_ACTIONS, actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=20,
            match=match, instructions=inst, buffer_id=msg.buffer_id)
        datapath.send_msg(mod)

    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath,
        in_port=in_port, actions=actions)
    datapath.send_msg(out)
```

Esercizio 2

Un sistema di blocco di URL fraudolente è costituito da un filtro di Bloom seguito da una hash table. Tutte le URL fraudolente sono inserite sia nel filtro di Bloom che nella hash table. Quando si tenta di accedere a una URL, si verifica se essa è presente nel filtro di Bloom. In caso positivo, allora si verifica se è presente anche nella hash table. Se la URL è presente sia nel filtro di Blomm sia nella hash table allora la URL viene bloccata. Altrimenti URL non è bloccata e si procede normalmente.

Il filtro di Bloom è costituito da un array di $m = 2^{14}$ bit e $k = 3$ funzioni hash. La hash table è costituita da un array di $\mu = 2^{10}$ elementi. In caso di collisione, le URL sono inserite in una lista.

7. Calcolare la probabilità di falso positivo a valle del filtro di Bloom dopo aver effettuato l'inserimento di 10 000 URL fraudolente.

Soluzione:

$$\left(1 - \exp\left(-\frac{kn}{m}\right)\right)^k = 59\%$$

8. Calcolare la probabilità di falso positivo a valle della tabella hash dopo aver effettuato l'inserimento di 10 000 URL fraudolente.

Soluzione: 0

9. Calcolare il numero medio di URL inserite in ogni elemento della hash table.

Soluzione:

$$\frac{n}{\mu} = 9{,}7$$

10. Calcolare il numero medio di accessi in memoria per verificare che una URL non sia fraudolenta.

Soluzione: Un singolo accesso per il filtro di Bloom. Con probabilità $(1 - 59\%)$ niente altro.

Con probabilità 59% dobbiamo guardare anche la hash table, con in media $1 + 9{,}7$ accessi.

Mettendo insieme le due cose abbiamo:

$$1 + 0{,}59 \times 10{,}7 = 7{,}3$$

11. Dimensionare m in modo da dimezzare il numero medio di accessi in memoria (per la cascata filtro + tabella).

Soluzione: Con $m = 2^{15}$ la probabilità di falso positivo scende al 21%, per cui il numero di accessi scende a 3,3.

Esercizio 3

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Destination Port
32.0.0.0/5	44.0.0.0/6	80
44.0.0.0/6	248.0.0.0/5	*
64.0.0.0/3	32.0.0.0/5	123
*	96.0.0.0/3	22
248.0.0.0/5	64.0.0.0/3	8080
96.0.0.0/3	*	*

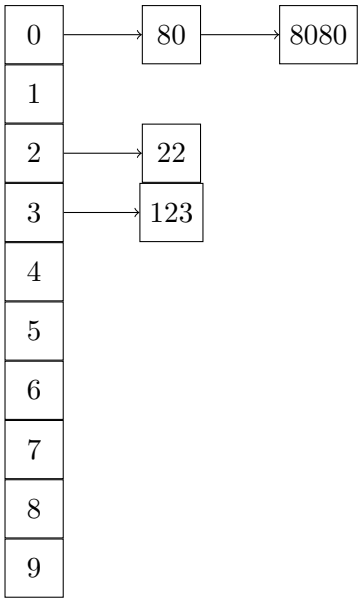
12. Definire il bitvector set per il campo porta destinazione.

Soluzione:

Destination Port	
22	010101
80	110001
123	011001
8080	010011

13. La ricerca della porta destinazione è implementata con una hash table usando la funzione hash $h(x) = x \bmod 10$. Disegnare la hash table e il suo contenuto.

Soluzione:



14. La ricerca del prefisso IP sorgente è implementato usando l'algoritmo di ricerca binaria in base alla lunghezza dei prefissi senza marcatori. Disegnare l'array corrispondente, indicando quali prefissi sono inseriti in quale elemento dell'array.

Soluzione:

ℓ	prefissi
1	
2	
3	64.0.0.0/3, 96.0.0.0/3
4	
5	32.0.0.0/5, 248.0.0.0/5
6	44.0.0.0/6
7	
...	

15. Considerare ora la presenza dei marcatori. Dire quali marcatori devono essere inseriti e in quale posizione. *Si consideri la ricerca a partire dalla lunghezza* $\ell = 16$.

Soluzione:

ℓ	prefissi
1	
2	marcatori per 64.0.0.0/2
3	64.0.0.0/3, 96.0.0.0/3
4	marcatori per 32.0.0.0/4, 240.0.0.0/4
5	32.0.0.0/5, 248.0.0.0/5
6	44.0.0.0/6
7	
8	Nessun marcatore necessario
...	
16	Nessun marcatore necessario
...	

16. Scrivere quali elementi dell'array vengono visitati durante la ricerca dell'indirizzo 44.0.0.1.

Soluzione:

ℓ	prefissi
16	nessun match
8	nessun match
4	match con il marker 32.0.0.0/4
6	match con 44.0.0.0/6
7	nessun match